Mini RISC ISA Proposal

Group 62

Allu Gowri Sasaank 24CS10054

Priyansh Pranjal 24CS10024

October 2026

1. Datapath and Architecture Images

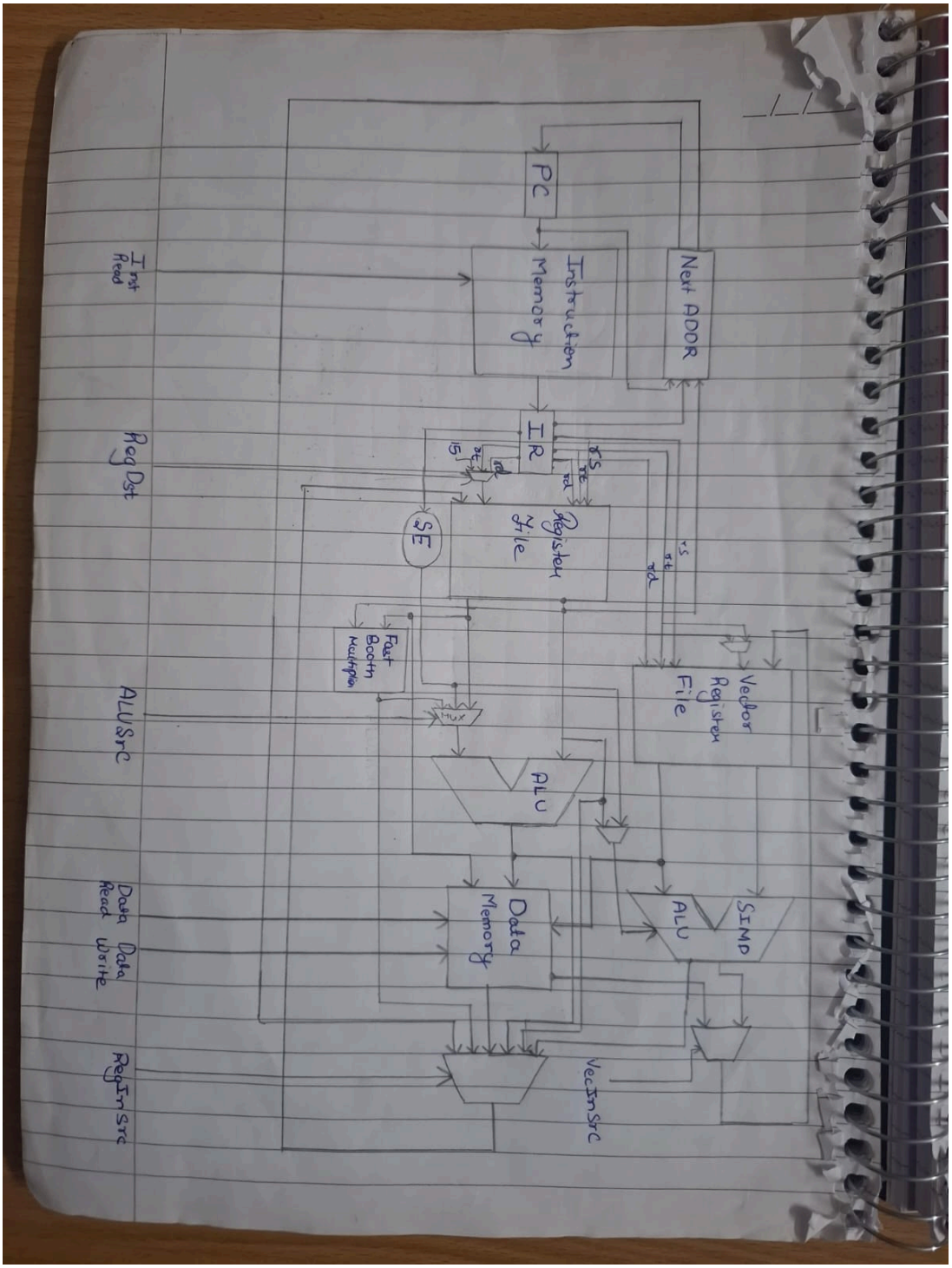


Figure 1: Complete Datapath

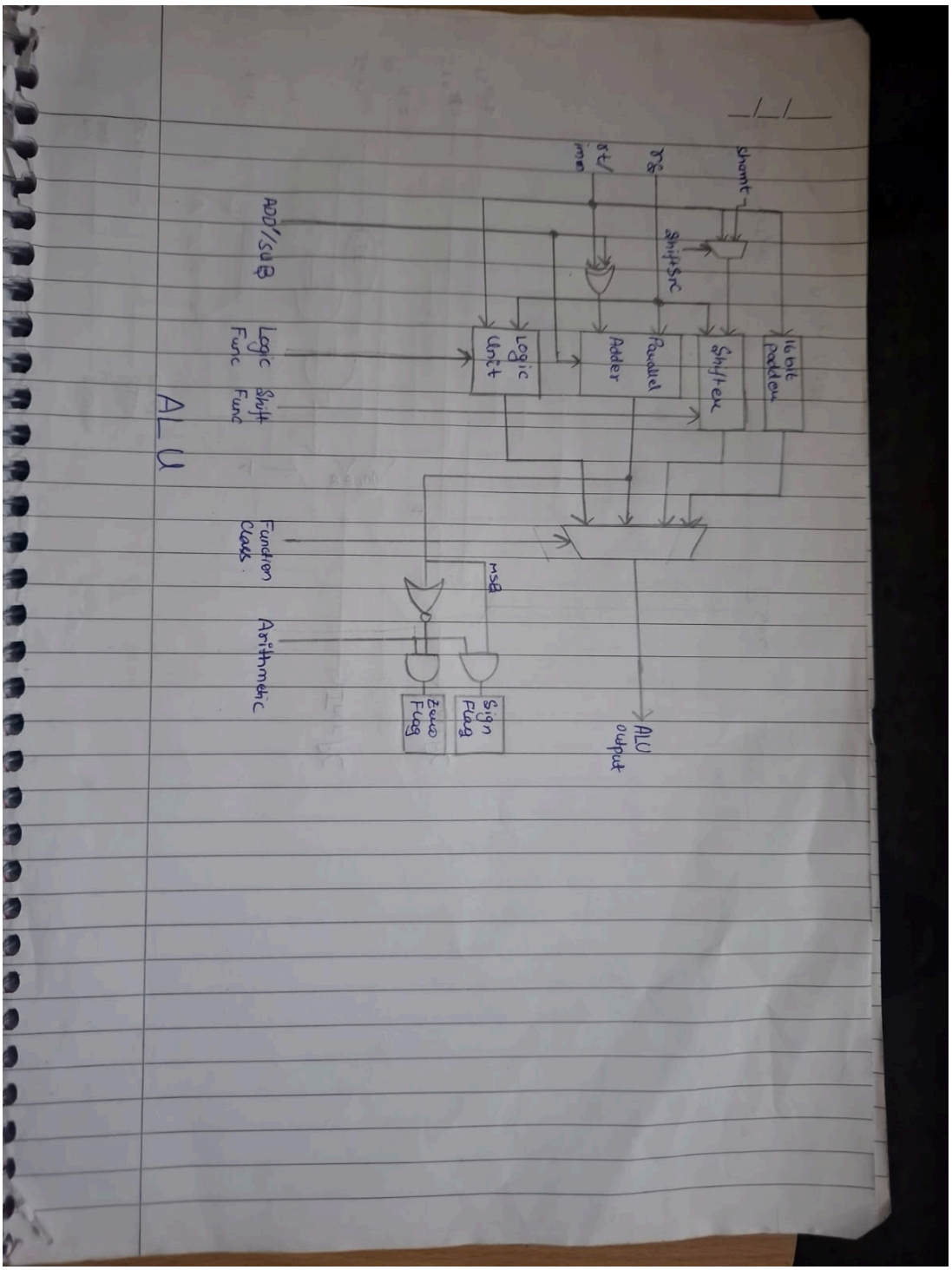


Figure 2: ALU Architecture

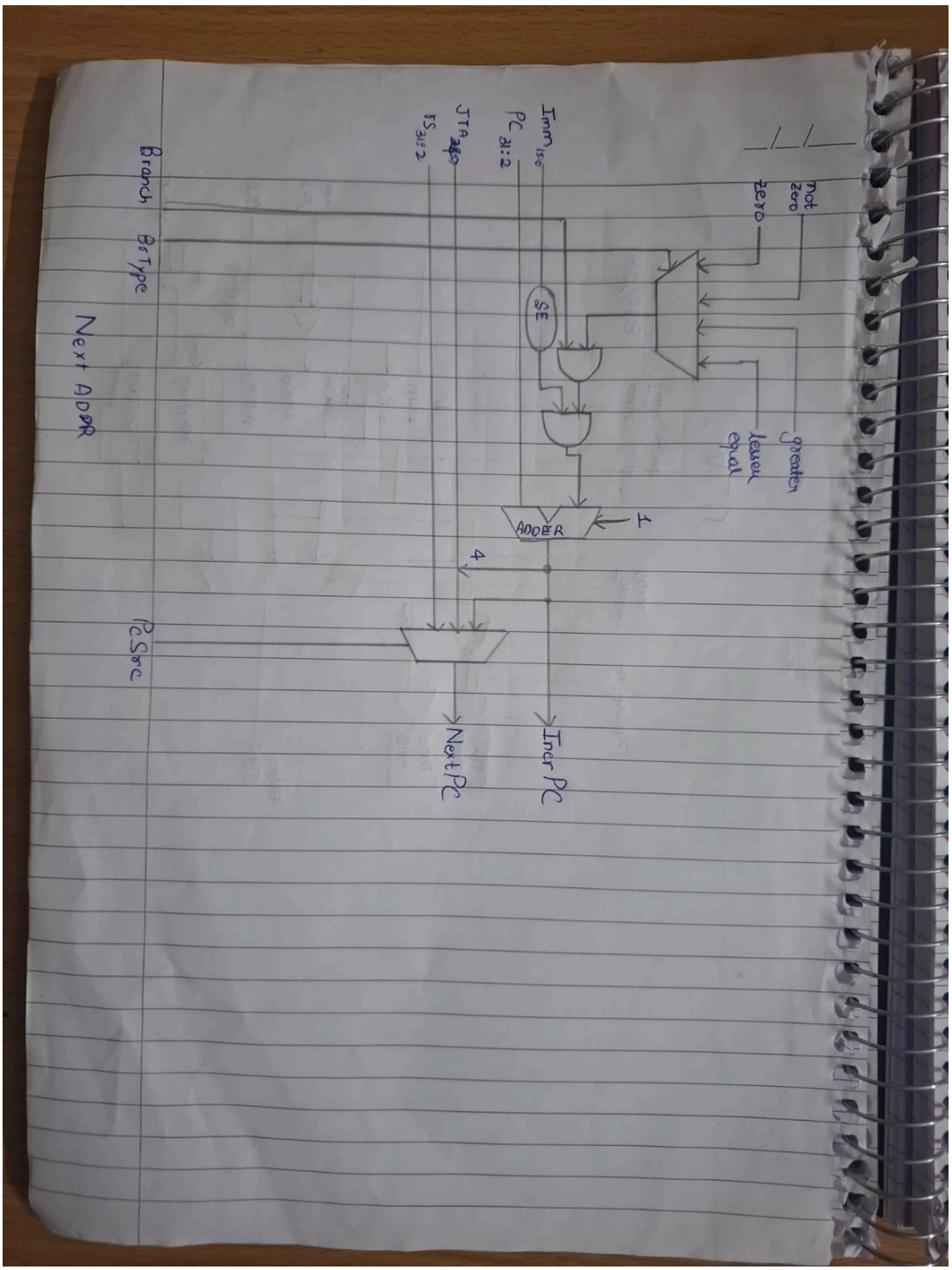


Figure 3: Next PC Logic

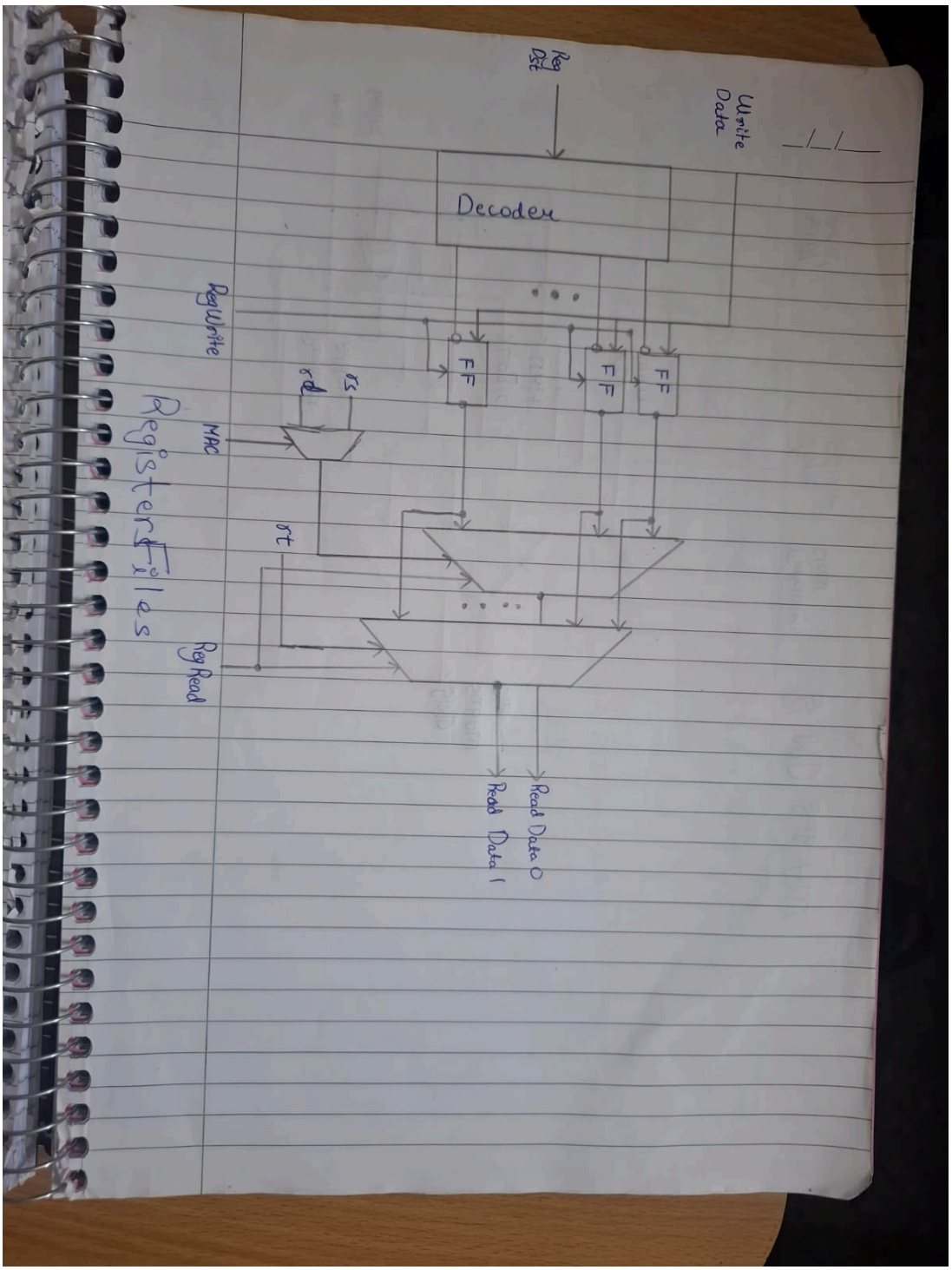


Figure 4: Register File

Figure 5: SIMD Sub ALU

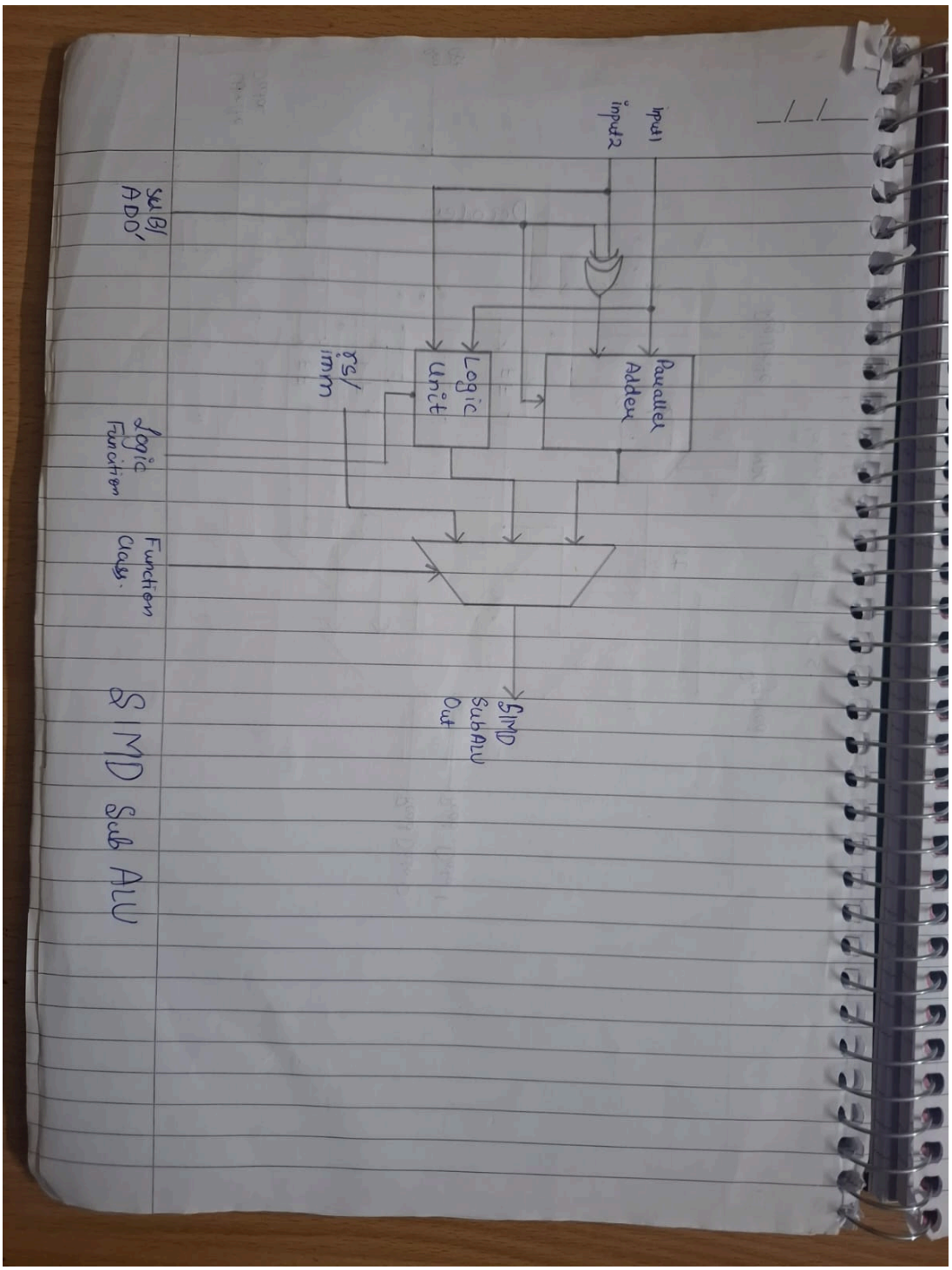


Figure 6: Overall SIMD ALU

2. Complete Instruction Set

2.1. ALU Instructions

Instruction	Syntax	Semantics
ADD	ADD rd, rs, rt	rd ← rs + rt
SUB	SUB rd, rs, rt	rd ← rs - rt

ADDI	ADDI rt, rs, imm	rt ← rs + imm
AND	AND rd, rs, rt	rd ← rs & rt
OR	OR rd, rs, rt	rd ← rs | rt
XOR	XOR rd, rs, rt	rd ← rs ^ rt
NOR	NOR rd, rs, rt	rd ← ~(rs | rt)
ANDI	ANDI rd, rs, imm	rt ← rs & imm
ORI	ORI rd, rs, imm	rt ← rs | imm
XORI	XORI rd, rs, imm	rt ← rs ^ imm
NORI	NORI rd, rs, imm	rt ← ~(rs | imm)
NOT	NOT rd, rs	rd ← ~rs
SLL	SLL rd, rs, shamt	rd ← rs << shamt
SRL	SRL rd, rs, shamt	rd ← rs >> shamt
SRA	SRA rd, rs, shamt	rd ← rs >> shamt (arithmetic/sign extended)
SLLV	SLLV rd, rs, rt	rd ← rs << rt[4:0]
SRLV	SRLV rd, rs, rt	rd ← rs >> rt[4:0]
SRAV	SRAV rd, rs, rt	rd ← rs >> rt[4:0] (arithmetic/sign extended)
MUL	MUL rd, rs, rt	rd ← rs * rt
MAC	MAC rd, rs, rt	rd ← rd + rs * rt
CMP	CMP rs, rt	rs - rt (sets flags, does not write back)
LUI	LUI rt, imm	rt ← imm << 16
CMOVZ	CMOVZ rd, rs	if Zero flag set, rd ← rs
CMOVN	CMOVN rd, rs	if Zero flag not set, rd ← rs
CMOVG	CMOVG rd, rs	if Zero flag not set and Sign flag not set, rd ← rs
CMOVL	CMOVL rd, rs	if Zero flag not set and Sign flag set, rd ← rs

2.2. Memory Instructions

Instruction	**Syntax**	**Semantics**

LB	LB rt, imm(rs)	rt ← Memory[rs + imm] (byte)
LW	LW rt, imm(rs)	rt ← Memory[rs + imm] (word)
LV	LV rt, imm(rs)	rt ← Memory[rs + imm] (vector)
SB	SB rt, imm(rs)	Memory[rs + imm] ← rt (byte)
SW	SW rt, imm(rs)	Memory[rs + imm] ← rt (word)
SV	SV rt, imm(rs)	Memory[rs + imm] ← rt (vector)
LBI	LBI rt, imm(rs)	rt ← Memory[rs + imm] (byte); rs ← rs + 1
LWI	LWI rt, imm(rs)	rt ← Memory[rs + imm] (word); rs ← rs + 4
SBI	SBI rt, imm(rs)	Memory[rs + imm] ← rt (byte); rs ← rs + 1
SWI	SWI rt, imm(rs)	Memory[rs + imm] ← rt (word); rs ← rs + 4

2.3. Branch/Jump Instructions

Instruction	**Syntax**	**Semantics**
J	J offset	PC ← (PC + 4)[31:24] | offset[23:0]
JR	JR rs	PC ← rs
JAL	JAL offset	ra ← PC + 4; PC ← (PC + 4)[31:24] | offset[23:0]
BZ	BZ imm	if Zero flag set, PC ← PC + 4 + imm
BN	BN imm	if Zero flag not set, PC ← PC + 4 + imm
BG	BG imm	if Zero flag not set and Sign flag not set, PC ← PC + 4+ imm
BL	BL imm	if Zero flag not set and Sign flag set, PC ← PC + 4 + imm
DECJ	DECJ rt, imm	rt ← rt - 1; if Zero flag not set, PC ← PC + 4 + imm

2.4. Program Control Instructions

Instruction	**Syntax**	**Semantics**
NOP	NOP	Does nothing
HALT	HALT	Stops the execution

2.5. SIMD Instructions

Instruction	Syntax	Semantics
SPLAT	SPLAT vd, rs	vd (all 4 32-bit parts) ← rs
SPLATI	SPLATI vd, imm	vd (all 4 32-bit parts) ← imm
ADDV	ADDV vd, vs, vt	vd ← vs + vt (4 operations in parallel)
SUBV	SUBV vd, vs, vt	vd ← vs - vt (4 operations in parallel)
ANDV	ANDV vd, vs, vt	vd ← vs & vt (4 operations in parallel)
ORV	ORV vd, vs, vt	vd ← vs | vt (4 operations in parallel)
XORV	XORV vd, vs, vt	vd ← vs ^ vt (4 operations in parallel)
NORV	NORV vd, vs, vt	vd ← ~(vs | vt) (4 operations in parallel)
MULV	MULV vd, vs, vt	vd ← vs * vt (4 operations in parallel)
MACV	MACV vd, vs, vt	vd ← vd + vs * vt (4 operations in parallel)
REDS	REDS rd, vs	rd ← sum(vs) (reduction with sum)
REDA	REDA rd, vs	rd ← bitwise_AND(vs) (reduction with AND)
REDO	REDO rd, vs	rd ← bitwise_OR(vs) (reduction with OR)
REDX	REDX rd, vs	rd ← XOR(vs) (reduction with XOR)
REDN	REDN rd, vs	rd ← NOR(vs) (reduction with NOR)

3. Instruction Formats

The ISA uses a fixed 32-bit instruction length. There are three primary instruction formats: R-type, I-type, and J-type. For `NOP` and `HALT` instructions, apart from the 8-bit opcode, the rest of the 24 bits are ignored by the datapath.

3.1. R-type Format

Used for register-to-register operations. **Instructions:** ADD, SUB, AND, OR, XOR, NOR, NOT, SLL, SRL, SRA, SLLV, SRLV, SRAV, MUL, MAC, CMP, CMOVZ, CMOVN, CMOVG, CMOVL, JR, ADDV, SUBV, ANDV, ORV, XORV, NORV, MULV, MACV, SPLAT, REDS, REDA, REDO, REDX, REDN.

opcode	rs	rt	rd	shamt	funct
8 bits	4 bits	4 bits	4 bits	5 bits	7 bits

- **opcode**: Primary opcode (e.g., distinguishing standard ALU vs SIMD vs mixed).
- **rs, rt**: Source registers.
- **rd**: Destination register.
- **shamt**: Shift amount (used for shift instructions).

- **funct**: Function code to specify the exact operation (e.g., ADD vs SUB) when the opcode groups them.

3.2. I-type Format

Used for operations with an immediate value, memory load/store, and conditional branches.
Instructions: ADDI, ANDI, ORI, XORI, NORI, LUI, LB, SB, LW, SW, LV, SV, LBI, SBI, LWI, SWI, DECJ, SPLATI.

opcode	rs	rt	imm
8 bits	4 bits	4 bits	16 bits

- **opcode**: Primary opcode.
- **rs**: Source register (often base address or source operand).
- **rt**: Target register (destination for loads, source for stores/ALU).
- **imm**: 16-bit immediate value or offset.

3.3. J-type Format

Used for unconditional jumps and long-range conditional branches. **Instructions:** J, JAL, BZ, BN, BG, BL.

opcode	jta (Jump Target Address / Offset)
8 bits	24 bits

- **opcode**: Primary opcode.
- **jta**: 24-bit jump target address or offset.

4. Opcode Allocations

Note: This opcode allocation is subject to change to make the control path leaner. This is a preliminary layout to group similar instructions.

4.1. R-type Instructions

For R-type instructions, the primary `opcode` distinguishes between purely 32-bit operations, purely vector operations, and mixed operations. The `funct` field identifies the specific operation.

Opcode `0x00` (32-bit General Purpose Registers)

Instruction	Opcode	Funct
ADD	0x00	0x01
SUB	0x00	0x02
AND	0x00	0x03
OR	0x00	0x04
XOR	0x00	0x05
NOR	0x00	0x06
NOT	0x00	0x07
SLL	0x00	0x08
SRL	0x00	0x09
SRA	0x00	0x0A
SLLV	0x00	0x0B

SRLV	0x00	0x0C
SRAV	0x00	0x0D
MUL	0x00	0x0E
MAC	0x00	0x0F
CMP	0x00	0x10
CMOVZ	0x00	0x11
CMOVN	0x00	0x12
CMOVG	0x00	0x13
CMOVL	0x00	0x14
JR	0x00	0x15

Opcode `0x01` (128-bit Vector Registers)

Instruction	Opcode	Funct
ADDV	0x01	0x01
SUBV	0x01	0x02
ANDV	0x01	0x03
ORV	0x01	0x04
XORV	0x01	0x05
NORV	0x01	0x06
MULV	0x01	0x07
MACV	0x01	0x08

Opcode `0x02` (Mixed 32-bit and Vector Registers)

Instruction	Opcode	Funct
SPLAT	0x02	0x01
REDS	0x02	0x02
REDA	0x02	0x03
REDO	0x02	0x04
REDX	0x02	0x05
REDN	0x02	0x06

4.2. I-type Instructions

Each I-type instruction has a unique 8-bit opcode.

Instruction	Opcode	Instruction	Opcode
ADDI	0x10	LV	0x24
ANDI	0x11	SV	0x25
ORI	0x12	LBI	0x26
XORI	0x13	SBI	0x27
NORI	0x14	LWI	0x28
LUI	0x15	SWI	0x29
LB	0x20	DECJ	0x2A

SB	0x21	SPLATI	0x2B
LW	0x22		
SW	0x23		

4.3. Program Control Instructions

These instructions use a unique 8-bit opcode. They typically do not require additional operands.

Instruction	Opcode
NOP	0x3F
HALT	0x40

4.4. J-type Instructions

Each J-type instruction has a unique 8-bit opcode.

Instruction	Opcode
J	0x30
JAL	0x31
BZ	0x32
BN	0x33
BG	0x34
BL	0x35

5. Addressing Conventions

5.1. Immediate Addressing

For all I-type operations, the 16-bit immediate field is uniformly sign-extended to 32 bits. This applies to arithmetic operations, memory offsets, and logical operations alike. Sign-extending everything unconditionally without checking the type of instruction simplifies the control path and datapath.

5.2. Branch Addressing

Branch instructions use a PC-relative addressing scheme. The immediate/offset provided in the instruction is added to the current Program Counter (PC) to compute the next address (`PC ← PC + 4 + offset`). This keeps branches position-independent and efficient.

5.3. Memory Addressing

Memory operations (load/store) utilize an indexed addressing mode (or base + offset). The effective address is computed by adding the 16-bit immediate offset to the value in the base register `rs` (i.e., `Address = rs + imm`). This allows for efficient structural data access, stack manipulation, and array traversal. Additionally, auto-incrementing memory instructions (like `LWI`, `SWI`, `LBI`, `SBI`) update the base register `rs` with `rs + stride` (e.g. `rs ← rs + 4` for `LWI`) immediately after the access, heavily optimizing sequential memory operations.

6. Multiplier and MAC Unit (Fast Booth Multiplier)

The architecture includes a dedicated Fast Booth Multiplier to efficiently compute multiplication (`MUL`) and Multiply-Accumulate (`MAC`) operations.

Instruction Semantics:

- **MUL**: `rd ← rs * rt`
  Multiplies the contents of `rs` and `rt` and stores the 32-bit result in `rd`.
- **MAC**: `rd ← rd + rs * rt`
  Multiplies the contents of `rs` and `rt`, adds the current value of `rd` to the product, and writes the final sum back into `rd`.

Implementation Details: Because a fast Booth multiplier can take multiple cycles to evaluate the result, the operation must halt the standard pipeline progress.

- When a `MUL` or `MAC` instruction is decoded, the Control Path asserts a `MulStart` control signal.
- The Control Path then intentionally stalls the pipeline, halting instruction fetch and execution phases until the multiplication completes.
- The multiplier unit performs the multiplication operation asynchronously to the main single-cycle combinational ALU logic.
- Upon completion, the multiplier unit asserts the `MulDone` control signal. The Control Path receives this signal and un-stalls the pipeline.
- For the `MAC` instruction specifically, once the multiplication is complete, the `AluSrc` is selected as the multiplier branch. The ALU is then used to compute the sum of the original `rd` and the product (`rs * rt`), and the result is written back to the destination register `rd`.

7. Justification for the Chosen ISA

The proposed MiniRISC ISA is designed to balance simplicity, computational density, and scalability. The key justifications are:

1. **Simplicity and Lean Control Path**: Using standard 32-bit fixed-length instructions with only three primary formats (R, I, J) simplifies instruction fetching and decoding. The unified R-type opcode grouping (distinguished by the `funct` field) keeps the primary opcode space sparse and the control logic straightforward.
2. **Rich Conditional Execution**: The inclusion of multiple conditional move (`CMOVZ`, `CMOVN`, `CMOVG`, `CMOVL`) instructions drastically reduces the need for explicit branching. This mitigates jump overhead, increases code density, and makes the program control flow much easier to manage.
3. **Efficient Memory Access**: The addition of auto-incrementing load/store instructions (`LWI`, `SWI`, `LBI`, `SBI`) avoids issuing separate `ADDI` instructions during sequential array processing or stack pops/pushes, thereby increasing code density and performance.
4. **Integrated SIMD Support**: A robust suite of vector instructions ensures the processor can handle data-parallel workloads seamlessly, which is increasingly critical in modern workloads. Operations like reductions (`REDS`, `REDA`) map elegantly to 128-bit SIMD paradigms.
5. **Dedicated DSP-like Capabilities**: The Fast Booth Multiplier and the dedicated `MAC` instruction provide digital signal processing (DSP) level performance for mathematical and matrix operations, acting as a crucial component for any embedded or advanced computational system.